" Investigating Aminoacid Enrichments and Patterns in Phase-Separating Proteins: Understanding Biases in Liquid-Liquid Phase Separation"

-Computation User Guide-

Version Control

All programs and computational results were written and obtained using:

-Python **3.11**

-Spyder **5.4.3**

-Anaconda Distribution **2.5.2**

Anaconda Distribution with Spyder available here: https://www.anaconda.com/download

In this Guide, images are available for easy step by step reproduction and usage. These images are from **Spyder Integrated Development Environment**, other properly configured Integrated Development Environments running the same Python version can be used but are not covered visually by this guide, the computational logic still applies.

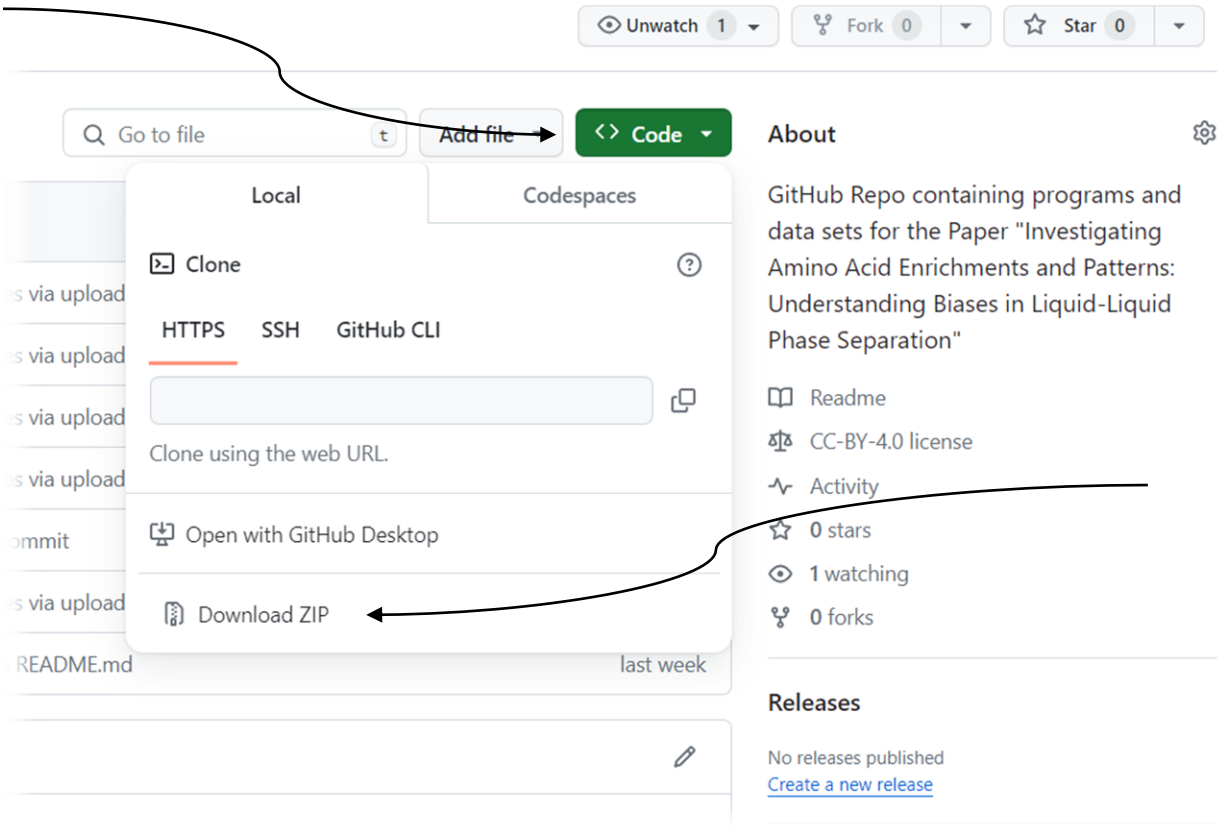
GitHub file access and download

The Folder, available at https://github.com/BPSlab/UnderstandingLLPS.git contains:

- Python programs used to archive computational results
- Datasets used to archive computational results
- Computational results
- User Guide to reproduce said computational results
- The paper "Investigating Amino Acid Enrichments and Patterns: Understanding Biases in Liquid-Liquid Phase Separation."

GitHub file access and download

After following the Link to the GitHub repo your navigator of choice should look like this:



Simply click the **Green Code button** and in the dropdown menu click **Download ZIP.**

GitHub file access and download

Extract the Downloaded ZIP file to your desired Location. The resulting folder is now ready to access, and it is referred in this guide as the **Main Project Directory** (Noted in Red):When opened, it presents the following structure:

Name	Date modified	Type	Size
Datasets	2/12/2024 12:20 PM	File folder	
Paper	2/6/2024 12:49 AM	File folder	
Programs	2/11/2024 10:56 PM	File folder	
Results	2/11/2024 9:35 PM	File folder	

In the following sections, each of these directories will be elaborated further.

File Structure

-Datasets-

This directory presents all Datasets used to reach the computational results that were not also themselves computational results from Python Programs. Below is a brief description of each program:

Protein Database: This file contains 198 phase-separating proteins, sourced from LLPSDB and PhaSePro. This curated database is the main focus of our study. This excel file is composed of two sheets: DPR and NODPR. Each sheet contains UniProt IDs, the respective protein sequence, the DPR or NODPR sequence, and protein function. The function(s) of each protein is marked with an 'X' in the respective function column.

Negative Database: This file contains 208 non-phase-separating proteins, sourced from UniProt. The excel file contains UniProt IDs and the respective protein sequence.

Families Data: This file contains the families corresponding to our 198 phase-separating proteins.

RNA binding/ DNA binding/Chromatin binding/Regulation/Hydrolase/Structure: These files are a variation of the *Protein Database*, each referring to a protein family selected for our study. Each file contains UniProt IDs, and corresponding DPR sequences.

Motif Picks/ Tripeptide Picks: These files contain motifs to be analyzed via python program.

File Structure

-Programs-

This directory contains all of the Programs used to obtain computational results. Below is a brief description of each script. Additional detailed usage will be present further in the document.

STATITIAN.py:

Input: Protein Database.xlsx, Negative Database.xlsx

Output: Protein Stats.xlsx, DPR Stats.xlsx, NODPR Stats.xlsx, Negative Stats.xlsx

Short Description: Takes input_dataframe, output_dataframe, output_file_name and control_variable, analyses sequences in input_dataframe, and saves them in output_dataframe. In the end, it outputs an excel file to the 'Results/Statistics' Folder, with output_dataframe information. This information encompasses the UniProt IDs, the sequence, number of amino acids in the sequence, amino acid count and percentage, amino acid side chain count and percentage, and families corresponding to the sequence.

PEPTITIAN.py:

Input: One of the following: DPR Stats.xlsx, RNA binding.xlsx, DNA binding.xlsx, Chromatin binding.xlsx, Regulation.xlsx, Hydrolase.xlsx, Structure.xlsx.

Output: FORMAT.xlsx in a Folder of user choice inside the Results Folder. 'FORMAT' corresponds to the input format of motifs being studied, for example:

Example for FORMAT = GXG:

Main Project Directory\Results\Motifs\Folder of user choice\GXG.xlsx

Short Description: Asks the User to insert the format of the motif to analyze in the form of a string with 'X' and 'Z'. 'X' is the position of the string that is Mutable, and 'Z' is the second input. Afterward, the program calls on user-defined functions to generate said motifs. For each motif, it goes through sequences in the Input Data frame and counts Motif presence and frequency.

FREQUENCY.py:

Input: Motif Picks.xlsx and **one** of the following: Protein Stats.xlsx **or** Negative Stats.xlsx

Output: Multiple MOTIF.xlsx Files in

(**Main Project Directory**\Results\Motif Presence)

One for each Motif in the Motif picks.xlsx File

Short Description:

For each motif in Motif Picks.xlsx, the program calls a Function that takes a motif and initializes two list variables. The first one is the list of the UniProt IDs of proteins that the motif is present, and the other is the list of the Uniprot IDs of Proteins that the motif is absent. It then cycles through the lines of either:

Protein Stats.xlsx **or** Negative Stats.xlsx

For each sequence, it counts Motif presence and Frequency. After the last sequence, it counts the number of UniProt IDs in each list and writes those values to the resulting Dataframe, under "In Uniprot" and "Not In Uniprot". Finally, it writes the resulting Dataframe to an excel file named after the motif.

MATRITIAN.py:

Input: Tripeptide Picks.xlsx **and** DPR Stats.xlsx

Output: Motif Occurrence Matrix.xlsx in

(**Main Project Directory**\Results\Matrix)

Short Description: Makes a Matrix with Rows and Columns, with values for each motif in Tripeptide Picks.xlsx. For each combination of motifs, it counts the occurrences of both motifs in all Sequences present in DPR Stats.xlsx.

File Structure

-Results-

In this directory are contained all computational results organized as follows:

Name	Date modified	Type	Size
Matrix	2/11/2024 9:35 PM	File folder	
Motif Presence	2/11/2024 6:27 PM	File folder	
Motifs	2/13/2024 8:20 PM	File folder	
Statistics	2/13/2024 7:38 PM	File folder	

Due to GitHub file restrictions, inside the **Motif Presence** and **Motifs** folders you will find several .ZIP Files for results.

MANUAL FILE SORTING IS ADVISED

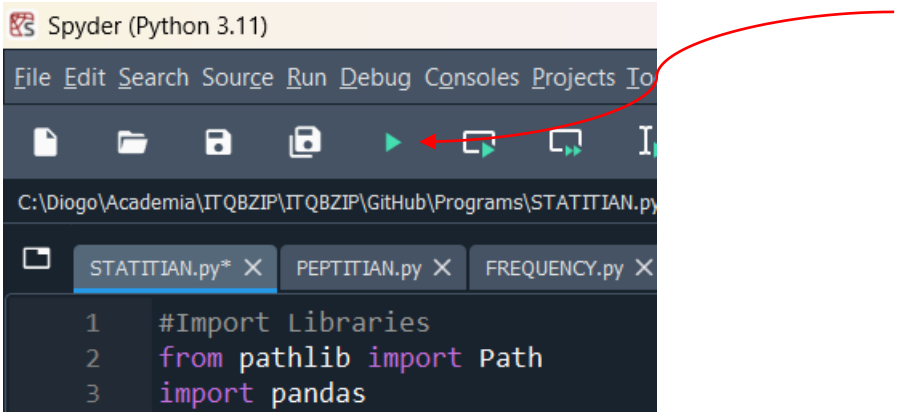
It is recommended to zip all MOTIF.xlsx files to an archive of your choosing.

Program Usage Guide

-STATITIAN-

For Regular usage:

-Run the Program



For Advanced usage:

-If you want to Run the program for different data sets you can easily do so if your files possess the same format as the ones used for the input, and you follow the next steps:

- Add a new Data frame from your desired Database excel

```
#Read Databases from Excel sheets
dprDataframe = pandas.read_excel((Path(mainProjectDirectory)/'Datasets'/'Protein Database.xlsx'),'DPR')
nodprDataframe = pandas.read_excel((Path(mainProjectDirectory)/'Datasets'/'Protein Database.xlsx'),'NODPR')
negativeDataframe = pandas.read_excel((Path(mainProjectDirectory)/'Datasets'/'Negative Database.xlsx'))
```

- Add a new Result Data frame

```
#Create statistics dataframes with columns named defined above
proteinStatisticsDataframe=pandas.DataFrame(columns=columnNamesList)
dprStatisticsDataframe=pandas.DataFrame(columns=columnNamesList)
nodprStatisticsDataframe=pandas.DataFrame(columns=columnNamesList)
negativeStatisticsDataframe=pandas.DataFrame(columns=columnNamesList)
```

-Add line in format:

statitian(input_dataframe,output_dataframe,output_file_name,control_variable)

input_dataframe is your Dataframe defined in the step above.

output_dataframe is your desired statistics Dataframe.

output_file_name is your desired output file name in string format.

control_variable is either "PROTEIN" or "SEQUENCE", which dictates if you run statistics in the full protein or just the sequence for files formatted like our Protein Database.xlsx

```
# Use of statitian function for data processing
statitian(dprDataframe,proteinStatisticsDataframe,"Protein Stats.xlsx","PROTEIN")
statitian(dprDataframe,dprStatisticsDataframe,"DPR Stats.xlsx","SEQUENCE")
statitian(nodprDataframe,nodprStatisticsDataframe,"NO DPR Stats.xlsx","SEQUENCE")
statitian(negativeDataframe,negativeStatisticsDataframe,"Negative Stats.xlsx","PROTEIN")
```

Program Usage Guide

-PEPTITIAN-

For Regular usage:

-Choose a Dataframe to search for motifs. The following Dataframes are already defined and ready to use:

dprStatisticsDataframe

rnaBindingDataframe

dnaStatisticsDataframe

chromatinbindingDataframe

regulationDataframe

hydrolaseDataframe

structureDataframe

Choose **one** and run main with the chosen one as entry value

(Example usage for dprStatisticsDataframe)

```
117
118 main(dprStatisticsDataframe)
119
```

-Run the Program, your terminal should look like this:

```
In [1]: runfile('C:/Diogo/Academia/ITQBZIP/ITQBZIP/GitHub/Programs/PEPTITIAN.py', wdir='C:/Diogo/Academia/
ITQBZIP/ITQBZIP/GitHub/Programs')
Enter the string with 'X' and 'Z' characters:
```

-Enter the Motif format. Valid examples are as follows: XZX, ZXXZ, XZZ, ...

```
In [1]: runfile('C:/Diogo/Academia/ITQBZIP/ITQBZIP/GitHub/Programs/PEPTITIAN.py', wdir='C:/Diogo/Academia/
ITQBZIP/ITQBZIP/GitHub/Programs')
Enter the string with 'X' and 'Z' characters: ZXZ
Enter the value of 'Z':
```

-Enter your Z value as a capital letter corresponding to an amino ccid. Valid examples are as follows: G, P, N, ...

```
In [1]: runfile('C:/Diogo/Academia/ITQBZIP/ITQBZIP/GitHub/Programs/PEPTITIAN.py', wdir='C:/Diogo/Academia/
ITQBZIP/ITQBZIP/GitHub/Programs')
Enter the string with 'X' and 'Z' characters: ZXZ
Enter the value of 'Z': G
Enter the name of the OutputFolder: |
```

- Enter your Desired Output Folder Name

- Due to GitHub file restrictions, inside of **Motif Presence** you will find several .ZIP Files for results computed with Protein Database and Negative, as such with each Run of the Program PEPTITIAN

MANUAL FILE SORTING IS ADVISED

It is recommended to zip all MOTIF.xlsx files to an archive of your choosing.

Program Usage Guide

-FREQUENCY-

For Regular usage:

-Choose in what Dataframe the motifs are to be counted. The following Dataframes are already defined and ready to use:

proteinStatisticsDataframe

negativeStatisticsDataframe

rnaBindingDataframe

dnaStatisticsDataframe

chromatinbindingDataframe

regulationDataframe

hydrolaseDataframe

structureDataframe

Choose **one** and run makeMotifList with the chosen one as entry value

(Example usage for proteinStatisticsDataframe)

```
makeMotifList(proteinStatisticsDataframe)#Change to respetive Dataframe to be analised
```

-Run the Program

For Advanced usage:

-If you want to Run the program for a different list of motifs simply replace the motifs in **Motif picks.xlsx**

Program Usage Guide

-MATRITIAN-

For Regular usage:

-Choose in what Dataframe motif ocurrences are to be counted. The following Dataframe are already defined and ready to use:

dprStatisticsDataframe

Choose one and run matrician with it as entry value

(Example usage for dprStatisticsDataframe)

```
matrician(dprStatisticsDataframe)
```

-Run the program

For Advanced usage:

-If you want to Run the program for a different list of motifs simply replace the motifs in **Tripeptide picks.xlsx**